

IERG 3080

Information and Software Engineering
Practice

< Group Project >
Simplified Taiko no Tatsujin Game

Group Member:

Yeung Man, Thomas (1155203181)

Wong Man Hei, Jimmy (1155213238)

Wong Tsz Hin, Alex (1155212117)

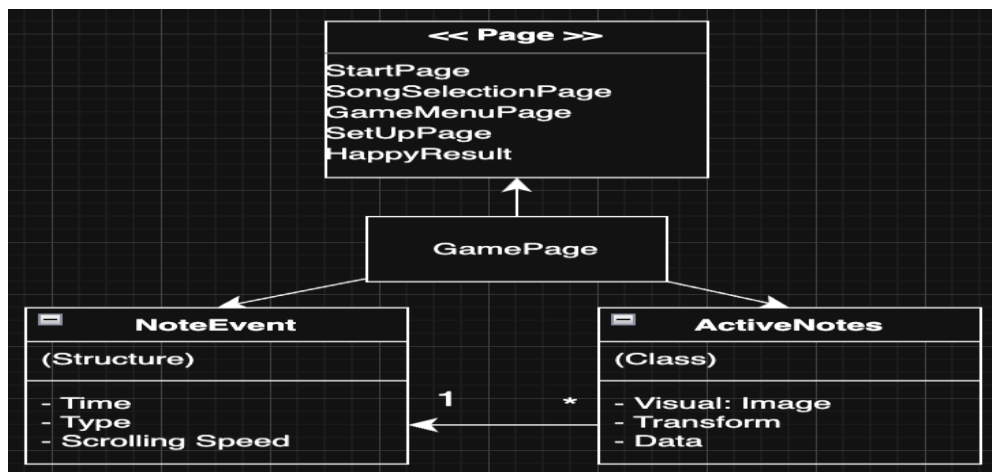
Due Date: 23-12-2025

Introduction

This project is a simplified single-player Taiko no Tatsujin rhythm game built using WPF and C#. Players press keys to hit scrolling red (Don) and blue (Ka) notes in time with the music. The game includes all required features: a title screen, song selection, gameplay with music synchronization, timing judgment (Perfect/Good/Bad), different hit sounds, scoring and combo system, and a results screen. It also adds the advanced features of difficulty selection (using different courses from a .tja file) and a settings page for customizing key bindings for both players and adjusting note scrolling speed (1×–4×), although simultaneous two-player mode was not implemented. Development followed Agile methodology with short iterative sprints and regular playtesting, focusing on precise timing and smooth WPF animations. This report describes the system structure with UML modeling, the Agile process, testing plan, and the main difficulties overcome.

System Structure and Design

The application follows a page-based navigation structure in WPF, with each major screen implemented as a separate Page class connected via NavigationService. This approach keeps screens independent while allowing smooth transitions between the title screen, song selection, difficulty selection, settings, gameplay, and results.



2.1 Overall Architecture

- **StartPage.xaml/cs:** Title screen with animated prompt.
- **SongSelectionPage.xaml/cs:** Song selection (currently one song, expandable design).
- **GameMenuPage.xaml/cs:** Difficulty selection screen.
- **SetupPage.xaml/cs:** Settings for key bindings (Player 1 and Player 2) and scrolling speed multiplier.

- **GamePage.xaml/cs:** Core gameplay logic, note management, input handling, timing judgment, and audio playback.
- **happyResult.xaml/cs:** Animated results screen showing score, max combo, and judgment counts.

2.2 Class Design (The main classes and why they exist)

- **NoteEvent**(a struct) stores pure data about each note: when it should appear (Time), what type it is (1 = red Don, 2 = blue Ka), and any special properties.
- **ActiveNote**(a class) combines the NoteEvent data with its on-screen visual (an Image) and movement (TranslateTransform). This links data to appearance.
- **GamePage** is the main controller — it loads the song, parses the .tja file, spawns notes, updates their positions every frame, handles key presses, calculates judgment, and manages score/combo.
- **Other Pages** (StartPage, SongSelectionPage, SetupPage, happyResult) handle different screens.

2.3 Class Reuse (all reused code or structures)

- All screens use WPF Page navigation instead of multiple windows — this reuses the same navigation system.
- Key bindings and scroll speed are stored as static properties in SetupPage, so any part of the game (now or in future multiplayer) can access the same settings without passing objects around.
- Note visuals reuse the same image assets (small/big face) and only change positions via TranslateTransform — no need to create new controls for each note.
- The result screen reuses the same XAML layout and just crops different parts of one sprite sheet (boygirl.png) to show Sad/Normal/Happy expressions.

2.4 Software Patterns Used (Name common patterns we applied)

- **Observer Pattern:** The game loop uses CompositionTarget.Rendering event — the system 'observes' each frame and updates all note positions automatically.
- **Singleton-like Global State:** Static properties in SetupPage act as a simple global configuration (common in games) for key bindings and speed.
- **Composition over Inheritance:** ActiveNote composes (contains) an Image and TranslateTransform rather than inheriting them — more flexible in WPF.
- **Code-behind with Separation of Concerns:** UI is in XAML, game logic and timing are in GamePage.cs, and pure data is in NoteEvent — keeping things organized even

without full MVVM.

Testing Plan

Main testing goals :

- Make sure notes appear and move at the correct time
- Check that hitting keys gives the right judgment (Perfect, Good, Bad, or Miss)
- Confirm score, combo, and hit sounds work properly
- Verify all screens (title, song selection, settings, gameplay, results) connect correctly

How I tested :

- **Functional testing:** Played the full song many times, trying to hit notes early, on time, late, or miss them on purpose.
- **Timing testing:** Tested at different scrolling speeds (1× to 4×) to ensure judgment stay accurate even when notes move faster.
- **Settings testing:** Changed key bindings and speed in the settings page, then checked that the game used the new settings correctly.
- **UI testing:** Resized the window and played to make sure everything scaled properly, and notes stayed in the right position.
- **Full playthrough testing:** Started from the title screen, selected song/difficulty, played the whole song, and checked that the results screen showed the correct score and counts.

Difficulties Overcome

The project faced two major challenges typical of rhythm games in WPF :

1. **Precise timing and synchronization** Standard timers in WPF were not accurate enough, causing judgment to feel inconsistent or delayed. **Solution:** Used `CompositionTarget.Rendering` for a high-precision game loop and calculate everything based on exact song time plus the `.tja OFFSET`, achieving reliable Perfect/Good/Bad judgment even at different scrolling speeds.
2. **Smooth, responsive note scrolling across window sizes and speeds** Notes became misaligned or jittery when resizing the window or using higher speed multipliers (up to 4×). **Solution:** Implemented `Viewbox` scaling for the playfield, dynamic position calculations on resize, and `TranslateTransform` for movement, ensuring notes always move smoothly and hit detection stay accurate.

Agile Methodology

Sprint	Duration	Main Goals and Features Delivered
1	Week 1–2	Title screen, song selection, difficulty selection, and basic page navigation
2	Week 3–4	Song playback, .tja file loading, note spawning, and basic scrolling
3	Week 5–6	Input handling, timing judgment (Perfect/Good/Bad), hit animations, and sound effects
4	Week 7–8	Scoring system, combo tracking, settings page (key binding and speed), and UI polishing
5	Week 9	Result screen with animations, bug fixes, final playtesting, and refinements

At the end of each sprint, the game was run and manually tested. Issues found were fixed immediately or added to the next sprint tasks. This incremental approach allowed early detection of problems (especially timing issues), frequent refactoring, and steady progress toward a complete, polished game.

Conclusion

As a solo developer, I applied Agile principles by breaking the project into short 1–2 week sprints, each producing a playable build with new features: the first sprint built the title, song selection, difficulty selection, and navigation screens; the second added song playback, .tja parsing, and basic note scrolling; the third implemented input handling, timing judgment, hit effects, and sounds; the fourth introduced scoring, combo tracking, the settings page for key bindings and speed adjustment, and UI polishing; and the final sprint completed the animated results screen, fixed remaining bugs, and performed final refinements. At the end of every sprint, I conducted manual playtesting, fixed issues immediately, and refactored code as needed, ensuring steady progress, early identification of timing problems, and a consistently improving gameplay experience.

Reference List

[1] Taiko Web Revival. "Taiko Web - A browser-based Taiko no Tatsujin simulator." Accessed December 23, 2025. <https://cjdgrevival.com/>. (Primary reference for gameplay mechanics and visual style.)

[2] Microsoft. "Windows Presentation Foundation documentation." Microsoft Learn. Accessed December 23, 2025. <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/>. (Official guideline for WPF development, XAML, animations, media playback, and UI controls.)

[3] "TJA File Format Specification." Community documentation (via Better Taiko Web project). Accessed December 23, 2025. <https://github.com/269Seahorse/Better-taiko-web/blob/master/TJA-format.mediawiki>. (Reference for parsing .tja chart files, including headers, courses, and note data.)

[4] EXILE TRIBE and community asset sources. "24karats TRIBE OF GOLD" (song) and Taiko no Tatsujin sprite rips from The Spriters Resource and similar free asset libraries. Accessed December 23, 2025. <https://www.spriters-resource.com/>. (Sources for music track, note/drum images, and other visual/audio assets used in the project.)